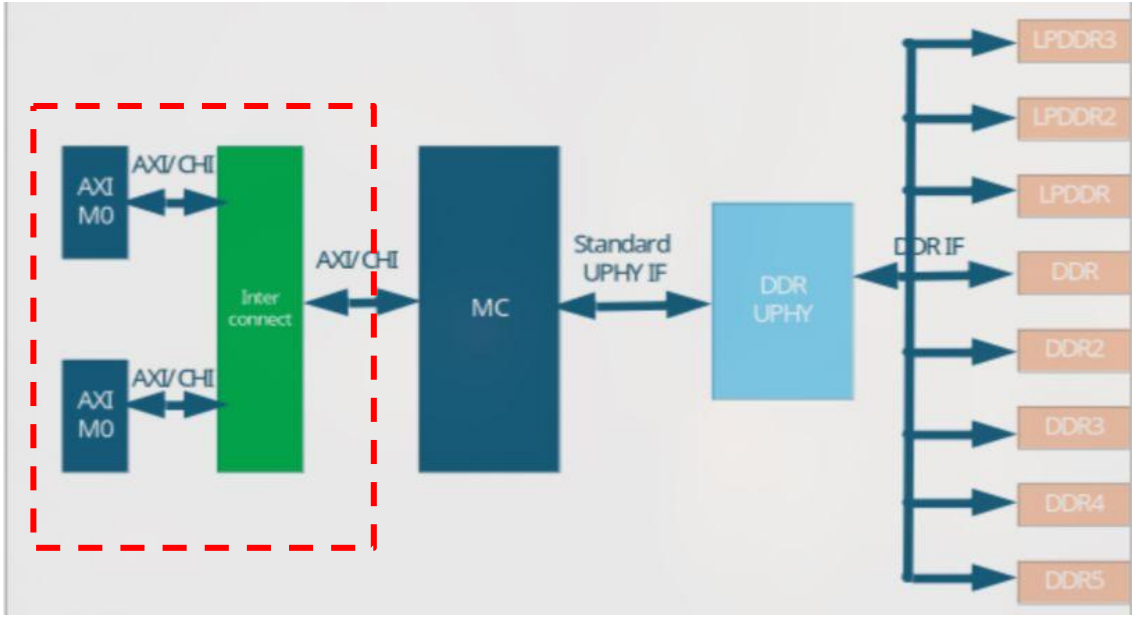
AXI Protocol

For High-Performance Memory Controllers

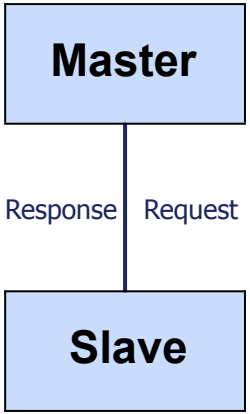


What is AXI?

AMBA Advanced eXtensible Interface

Design Goals:

- High bandwidth, low latency
- High-frequency operation
- Flexible (wide range of components)
- **Optimized for memory controllers** with high initial access latency



Key Innovation:
Burst-based transactions with out-of-order completion

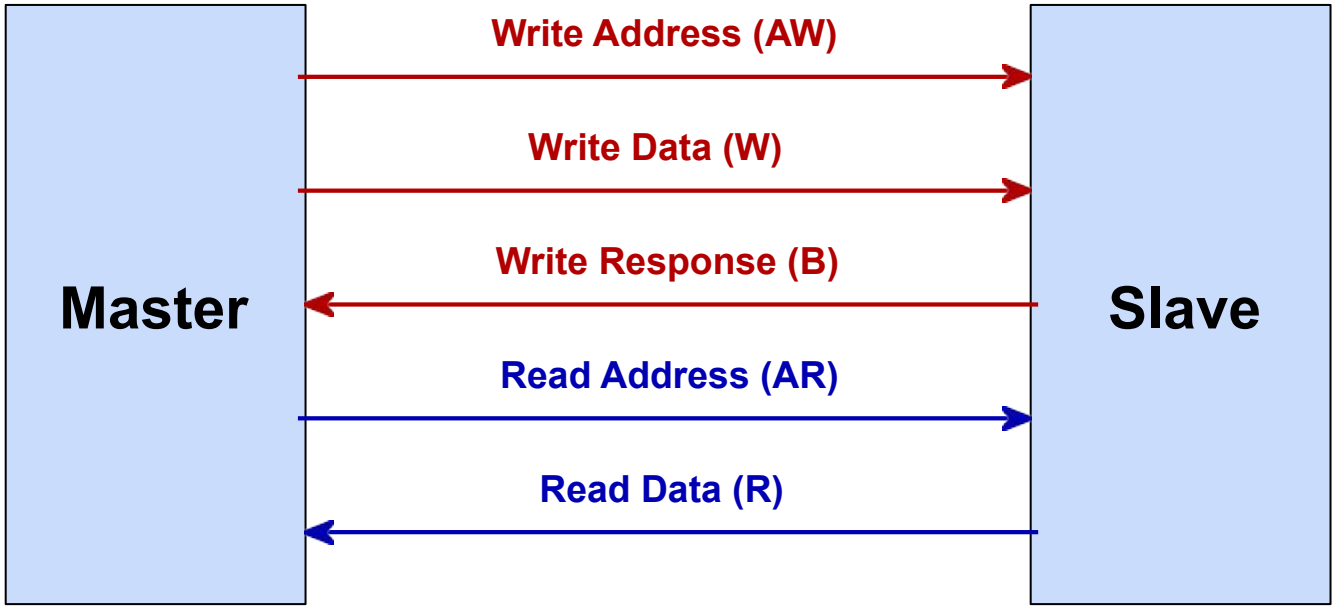
Perfect for DDR Memory!

AXI Protocol

Feature	AXI3	AXI4	AXI4-Lite
Burst Length	1-16 beats	**1-256 beats**	1 beat only
Write Interleaving	Yes (WID)	**No (removed)**	N/A
QoS Signaling	No	**Yes**	No
Region Signals	No	Yes	No
Use Case	Legacy	**Memory Ctrl**	Simple Periph

AXI4 recommended for new memory controller designs

Five Independent Channels



Key Benefit: Read and Write operate **simultaneously** (full duplex)

Write Transaction Channels

Write Address (AW)

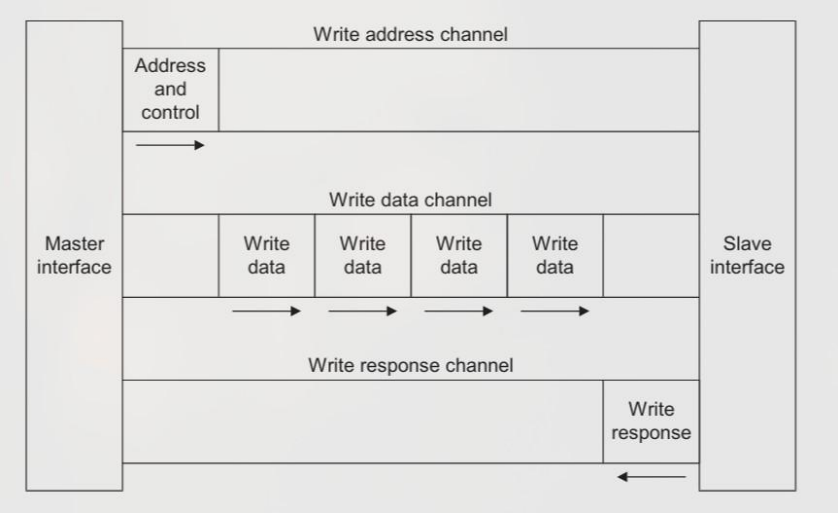
- AWID, AWADDR
- AWLEN, AWSIZE
- AWBURST, AWQOS

Write Data (W)

- WDATA, WSTRB
- WLAST

Write Response (B)

- BID, BRESP

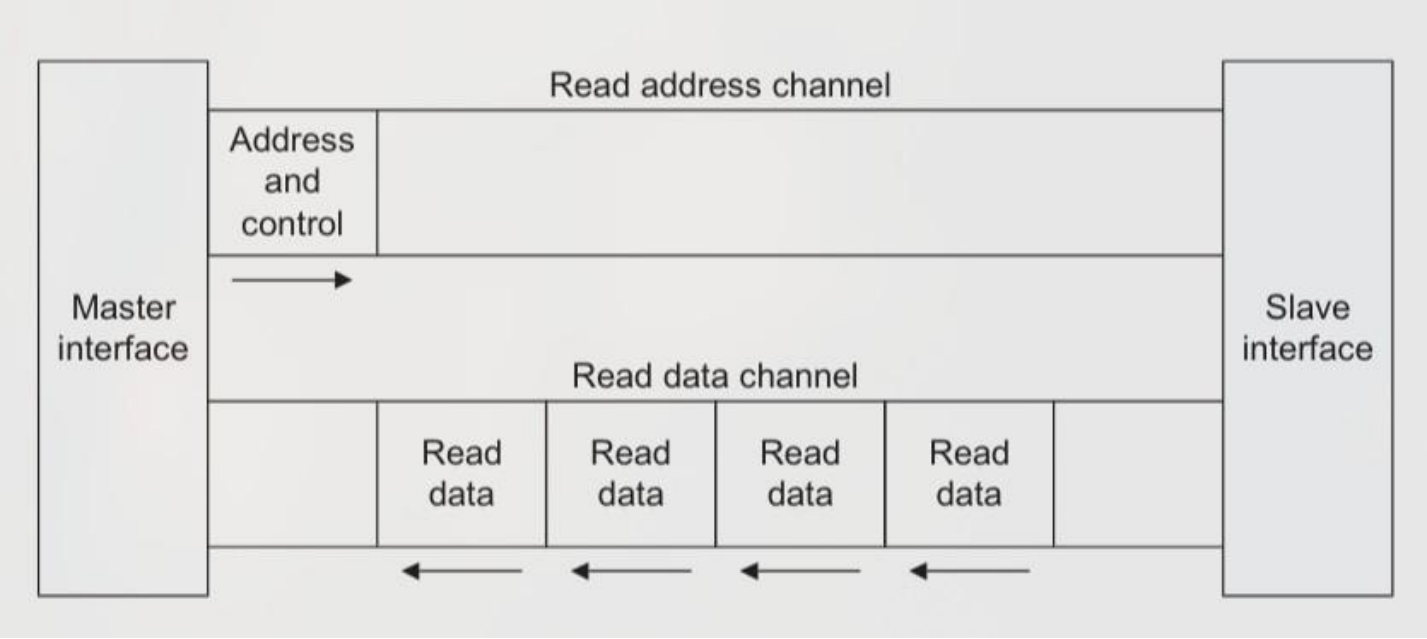


Read Transaction Channels

1. **Read Address (AR)**
   - ARID - Transaction ID
   - ARADDR - Start address
   - ARLEN - Burst length
   - ARSIZE - Bytes per beat
   - ARBURST - Type (FIXED/INCR/WRAP)
   - ARQOS - Priority (AXI4)

2. **Read Data (R)**
   - RID - Matches ARID
   - RDATA - Actual data
   - RRESP - Status
   - RLAST - Final beat indicator

Read Channel



VALID/READY Handshake Protocol

Every channel uses:

- **VALID** - Source has data
- **READY** - Destination can accept
- **Transfer** - Both HIGH

Critical Rules:

- VALID **must NOT** wait for READY
- READY **can** wait for VALID
- Prevents deadlock!

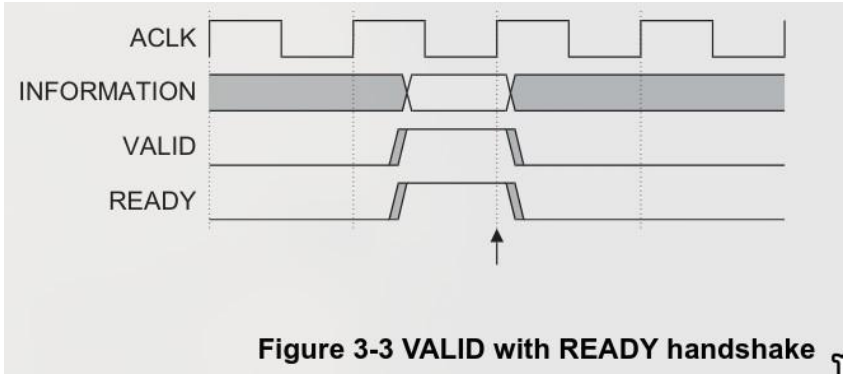


Figure 3-3 VALID with READY handshake

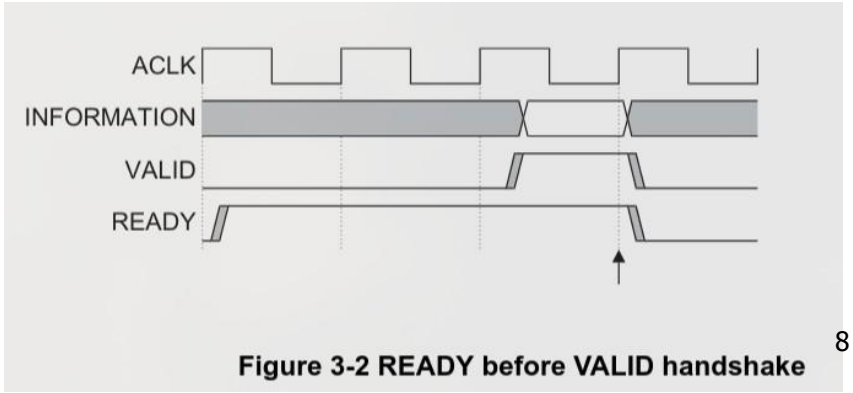


Figure 3-2 READY before VALID handshake

Burst Transaction Parameters

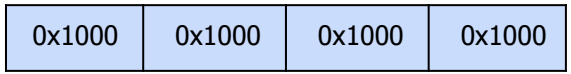
Parameter	Encoding	Meaning
AWLEN/ARLEN	0-255 (AXI4) 0-15 (AXI3)	Beats = LEN + 1
AWSIZE/ARSIZE	3 bits 000 → 1 byte 011 → 8 bytes 111 → 128 bytes	Bytes/beat = 2^{SIZE}

Total Burst Size = (AWLEN + 1) × 2^{AWSIZE}

AXI4 Max: 256 beats × 128 bytes = **32 KB** per burst!

Burst T

FIXED (00)

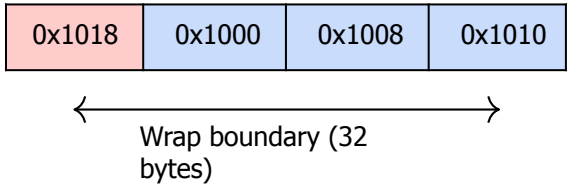


Address constant

INCR (01)

Increments by SIZE

WRAP (10)

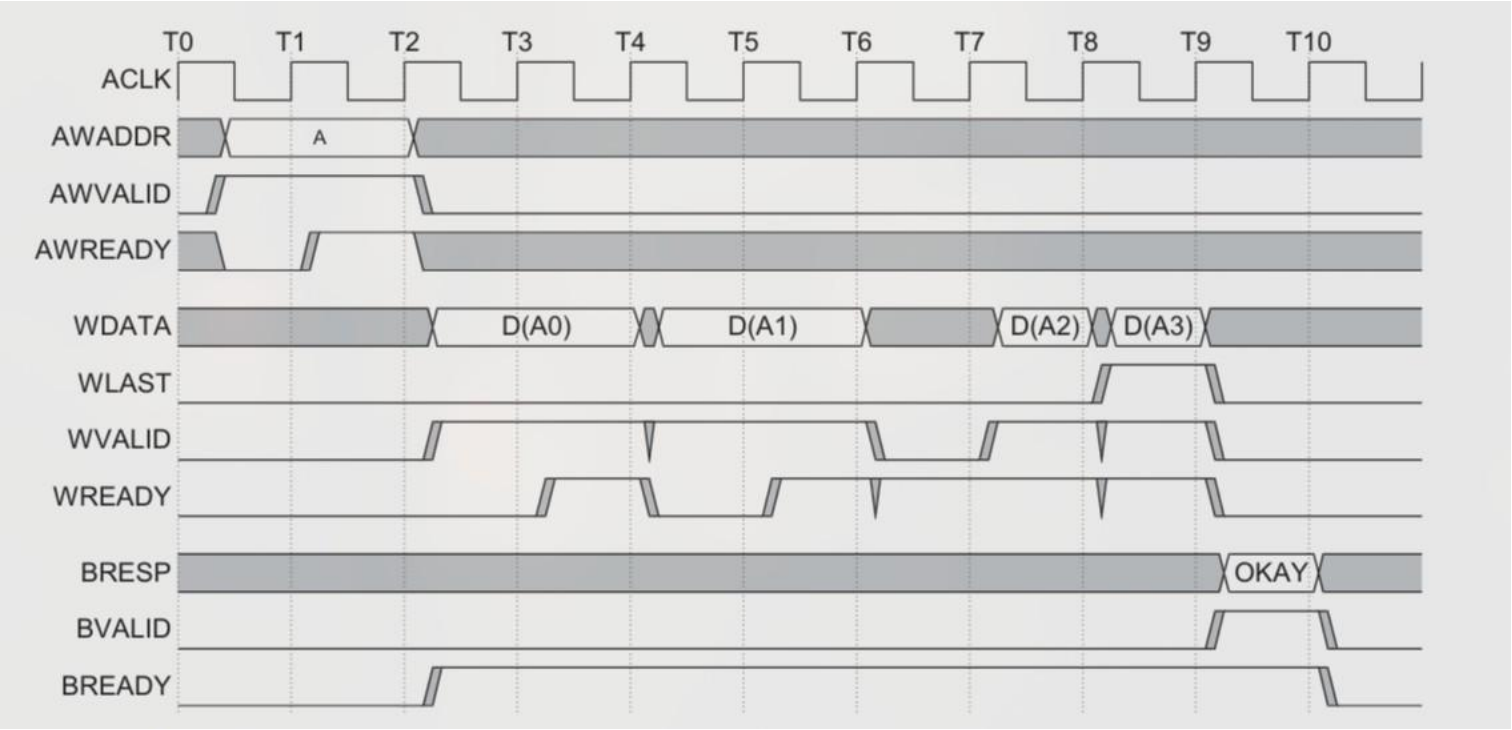


Wraps at boundary

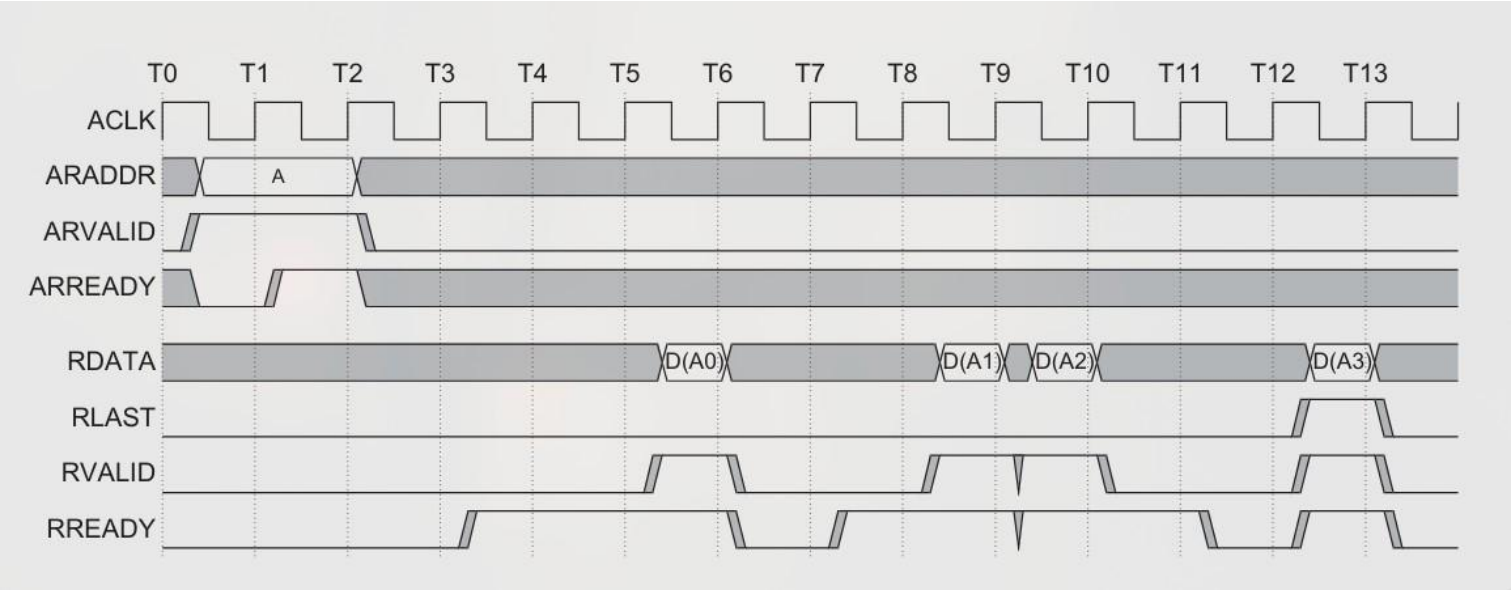
Most common for DDR:
INCR

For caches:
WRAP

Write Burst

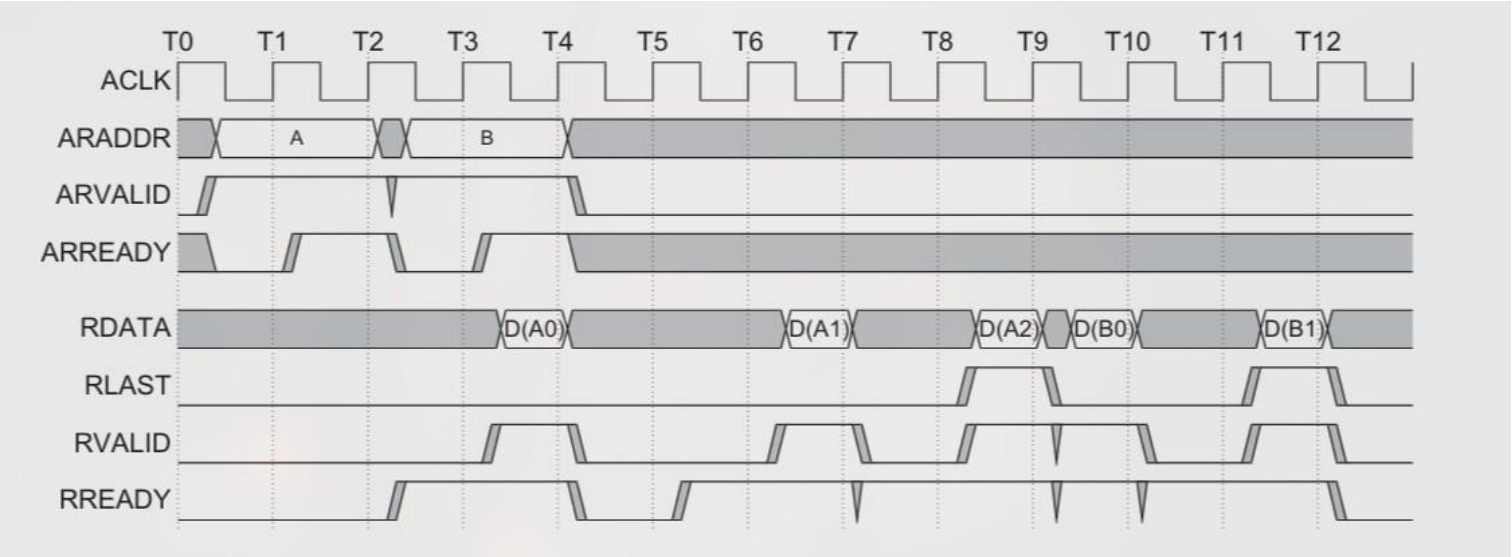

T0
T1
T2
T3
T4
T5
T6
T7
T8
T9
T10
ACLK
AWADDR
A
AWVALID
AWREADY
WDATA
D(A0)
D(A1)
D(A2)
D(A3)
WLAST
WVALID
WREADY
BRESP
OKAY
BVALID
BREADY

Read Burst



Slave returns multiple data beats for single address request

Overlapping Bursts



Multiple transactions in flight → Hides memory latency!

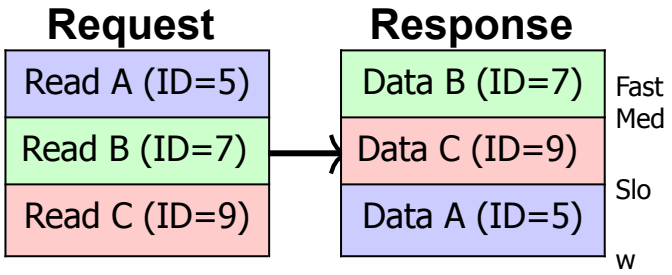
Transaction IDs & Out-of-Order

Why IDs?

- Memory has high latency
- Issue multiple requests
- Complete when ready
- Use ID to route response

Ordering Rules:

- **Same ID** → In-order
- **Different ID** → Any order



Out-of-order = Performance!

AXI4 Enhancements Over AXI3

Performance:

- **256 beat bursts** (was 16)
- Longer sequential access
- Better DDR utilization

Quality of Service:

- ARQOS/AWQOS signals
- 4-bit priority (0-15)
- Real-time traffic prioritization

Simplification:

- **Removed WID** signal
- No write data interleaving
- Simpler slave implementation
- Less hardware complexity

Additional:

- Region identifiers
- User-defined signals
- Enhanced cache attributes

AXI to DDR Command Translation

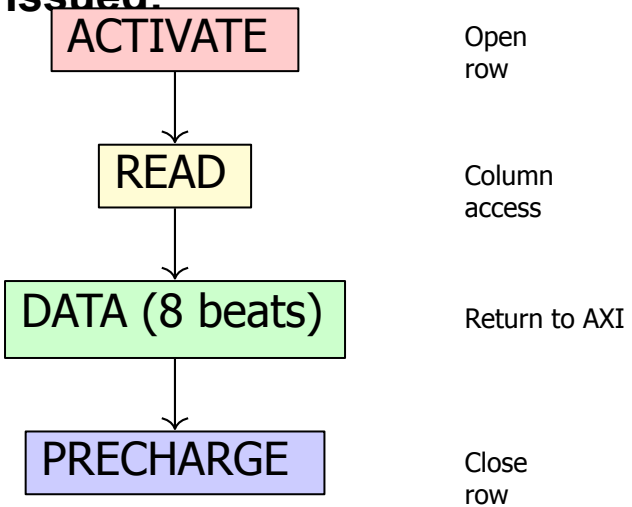
AXI Read Request:

- ARADDR = 0x9000 1000
- ARLEN = 7 (8 beats)
- ARSIZE = 3 (8 bytes)

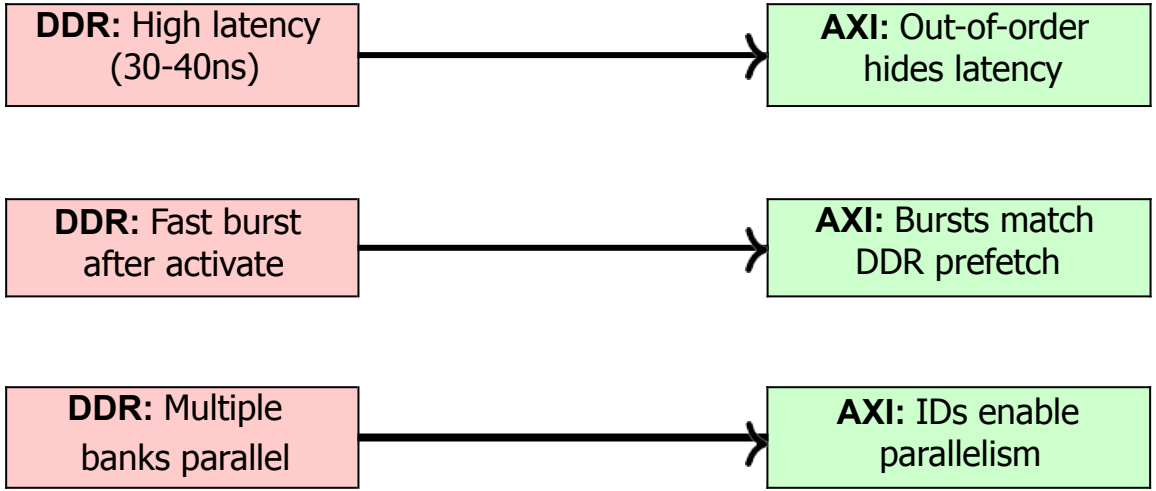
Memory Controller Decodes:

- Bank = 2
- Row = 0x900
- Column = 0x10

DDR Commands Issued:



Why AXI is Perfect for DDR Memory



Perfect match between protocol and memory!

Memory Controller Optimizations Using AXI

Transaction Reordering:

- Row buffer hits first
- Bank parallelism
- Min read/write switch

Write Buffering:

- Collect small writes
- Combine to DDR bursts
- Early response (BRESP)

QoS Scheduling:

- Priority-based (ARQOS/AWQOS)
- Real-time = high priority
- DMA = low priority

Burst Alignment:

- Match DDR BL8
- Align to cache lines
- Respect 4KB boundary

Key Takeaways

5 Independent Channels
Full duplex read/write

Burst-Based Protocol
One address, multiple data

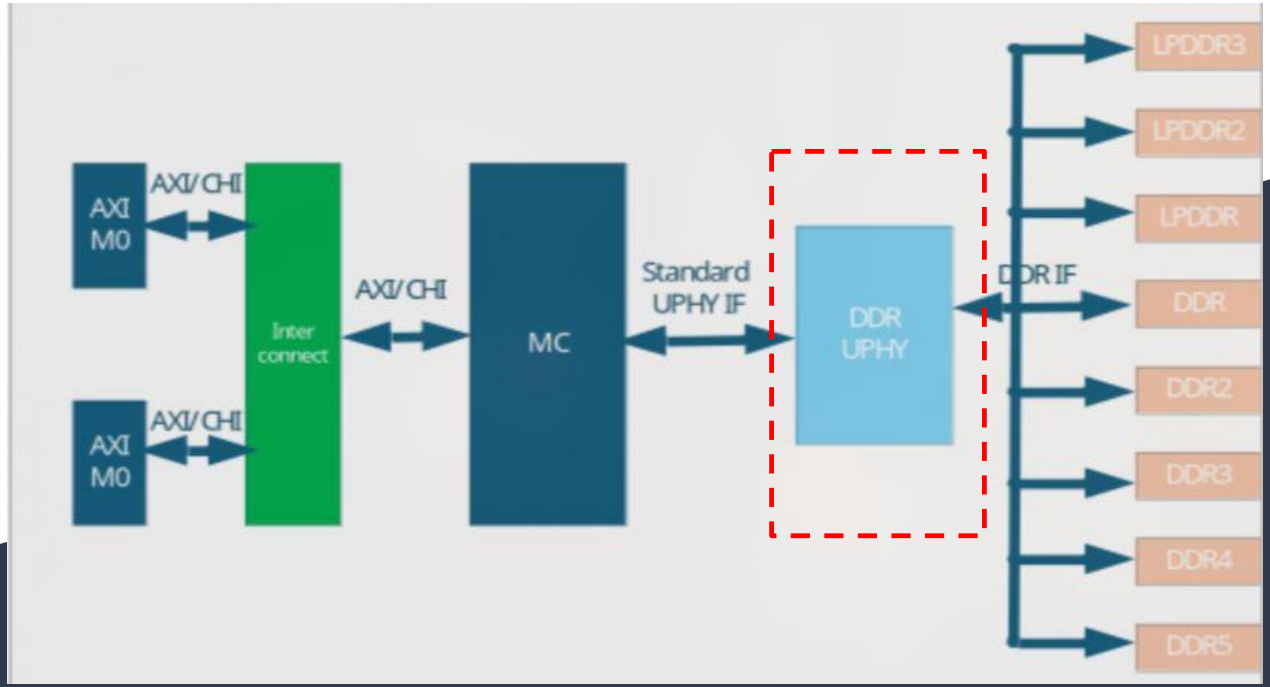
Out-of-Order Capable
Transaction IDs enable flexibility

Perfect for DDR
Hides latency, max- imizes bandwidth

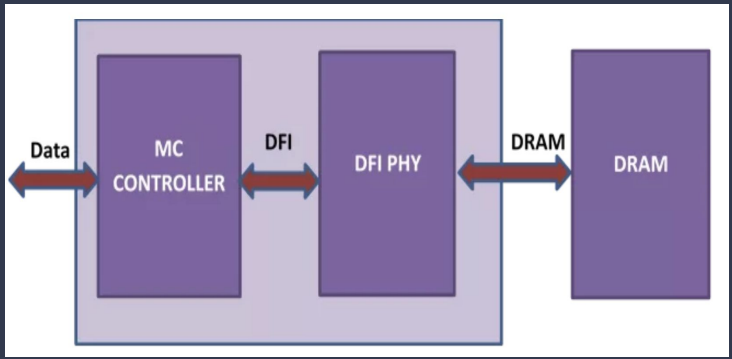
Critical Rules:

VALID must NOT wait for READY • Bursts cannot cross 4KB • Same ID = ordered

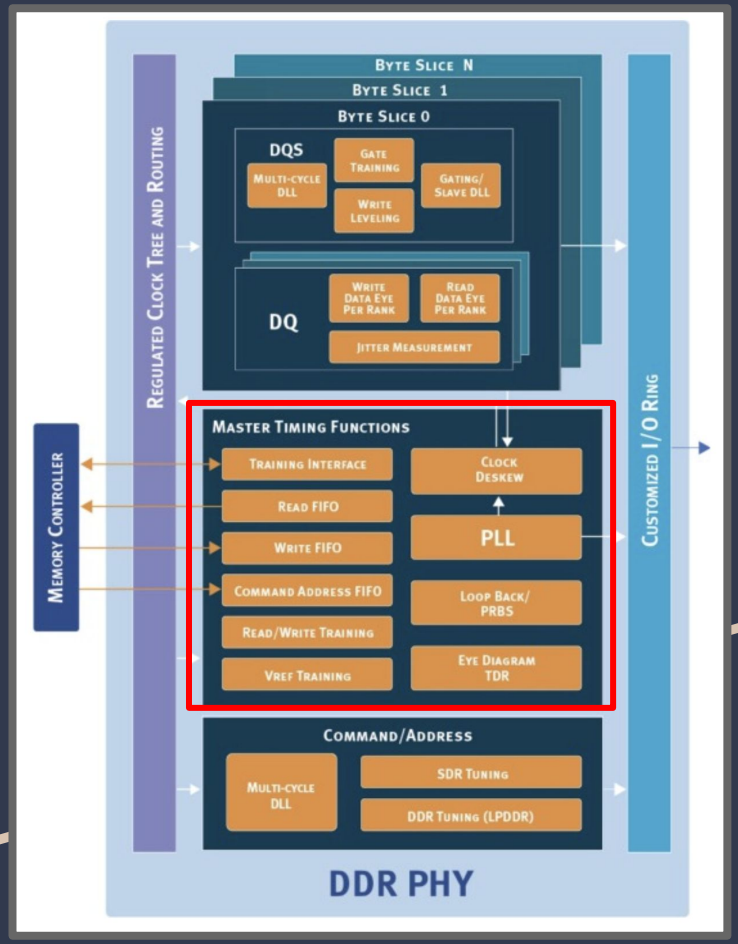
PHY Interfacing



What is PHY?

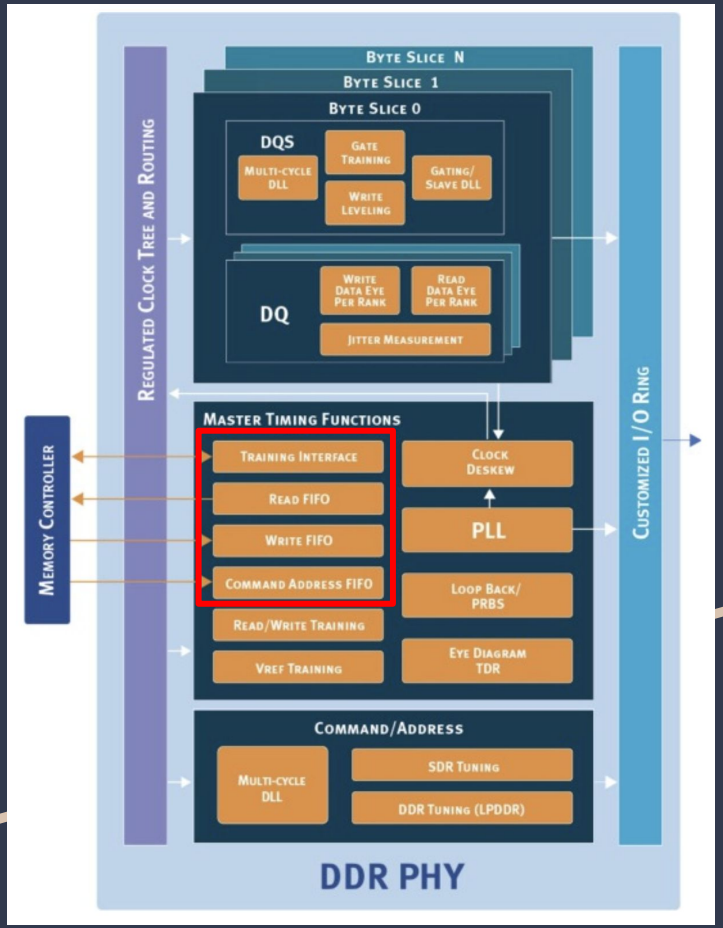


- Physical memory interface (PHY) for a memory controller
- It is a hardware that acts as a crucial bridge connecting the digital controller logic to the high-speed, external analog DRAM devices
- Translates logical memory commands into physical signals on the memory bus



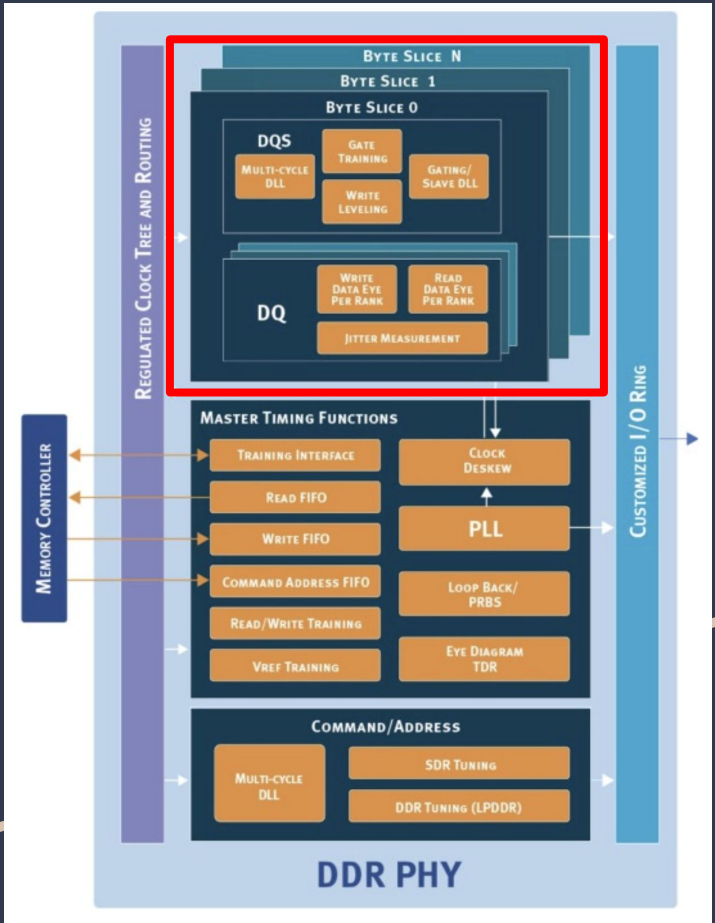
Master Timing Functions

- PLL (Phase-Locked Loop)
- Clock Deskew
- VREF Training
- Eye Diagram TDR
- Loop Back / PRBS



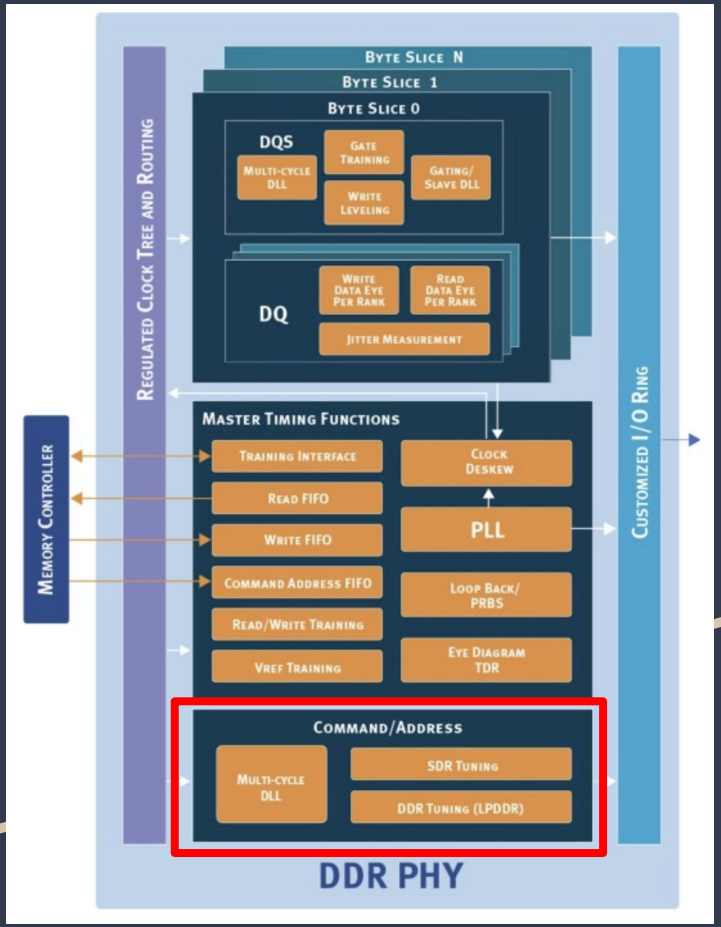
Memory Controller Interface

- Write FIFO
- Read FIFO
- Command Address FIFO
- Training Interface



Byte Slices

- **DQS (Data Strobe):** This acts as a "local clock" for the data.
  - Gate Training / Write Leveling
  - Multi-Cycle / Slave DLL
- **DQ (Data):** The actual bits of data.
  - Write/Read Data Eye Per Rank
  - Jitter Measurement



Centre for Heterogeneous and Intelligent Processing Systems
PES University | Electronic City Campus
www.chips.pes.edu

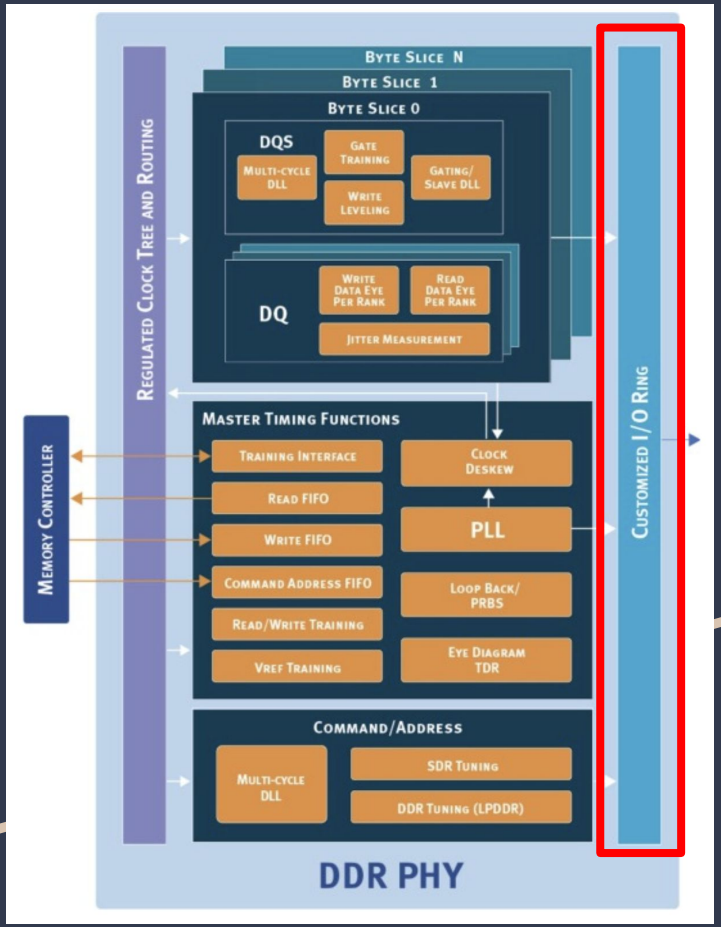
CPU GPU FPGA ASIC

Command/Address Block

- Multi-Cycle DLL
- SDR / DDR Tuning

Customized I/O Ring

- It contains the **analog drivers** and **receivers** that physically push electricity onto the PCB copper traces toward the RAM sticks. It handles **Impedance Matching (ZQ)** to prevent signal reflections.



Centre for Heterogeneous and Intelligent Processing Systems
PES University | Electronic City Campus
www.chips.pes.edu

Command/Address Block

- Multi-Cycle DLL
- SDR / DDR Tuning

Customized I/O Ring

- It contains the **analog drivers** and **receivers** that physically push electricity onto the PCB copper traces toward the RAM sticks. It handles **Impedance Matching (ZQ)** to prevent signal reflections.

PHY Control Block (PUB/PCTL)

brain or conductor of PHY

1. Writing Leveling
   - A series of "test pulses" are sent to the DRAM
   - **Delay-Locked Loops (DLLs)** are adjusted

2. DQS Gate training
   - PHY must ‘open gate’ when data is sent
   - If gate open is : Early -> electric noise captured
     Late -> misses the start of data

3. Vref training : finds perfect V_{th} maximizing data eye in the data eye diagram
   - Controller send a self generated data pattern to RAM
   - System sets the final V_{REF} to the **mathematical center** of the passing region, maximizing the noise margin

Data Eye Diagram

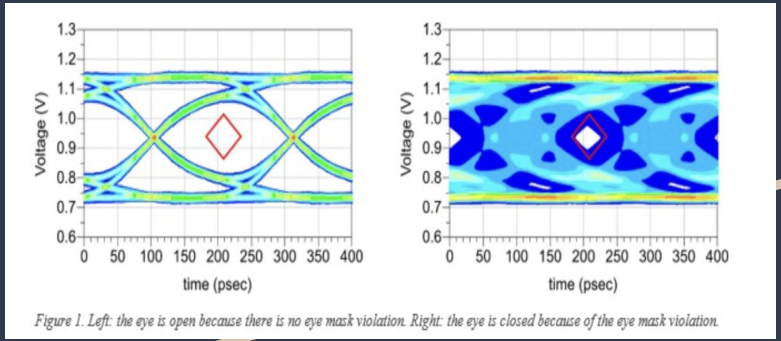
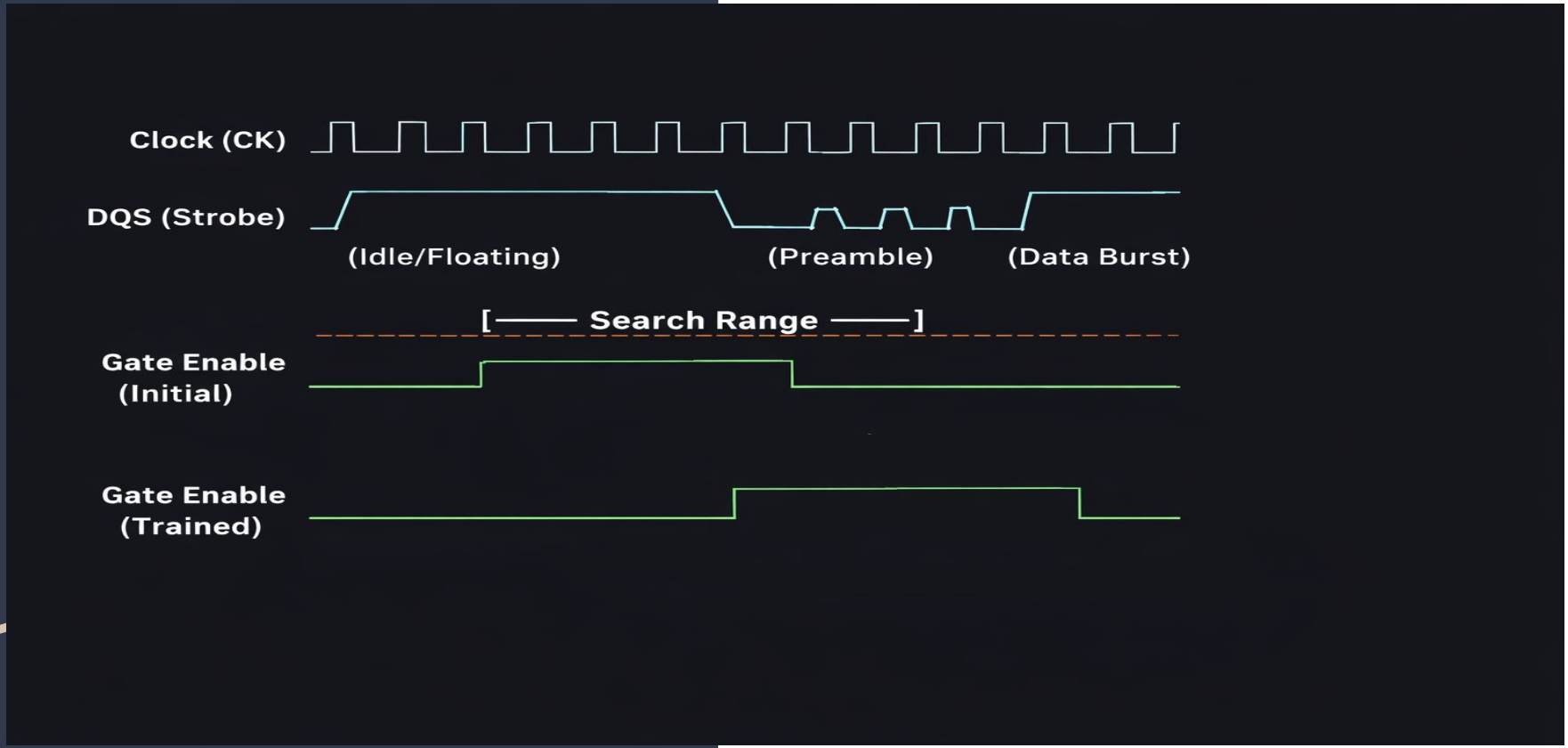


Figure 1. Left: the eye is open because there is no eye mask violation. Right: the eye is closed because of the eye mask violation.

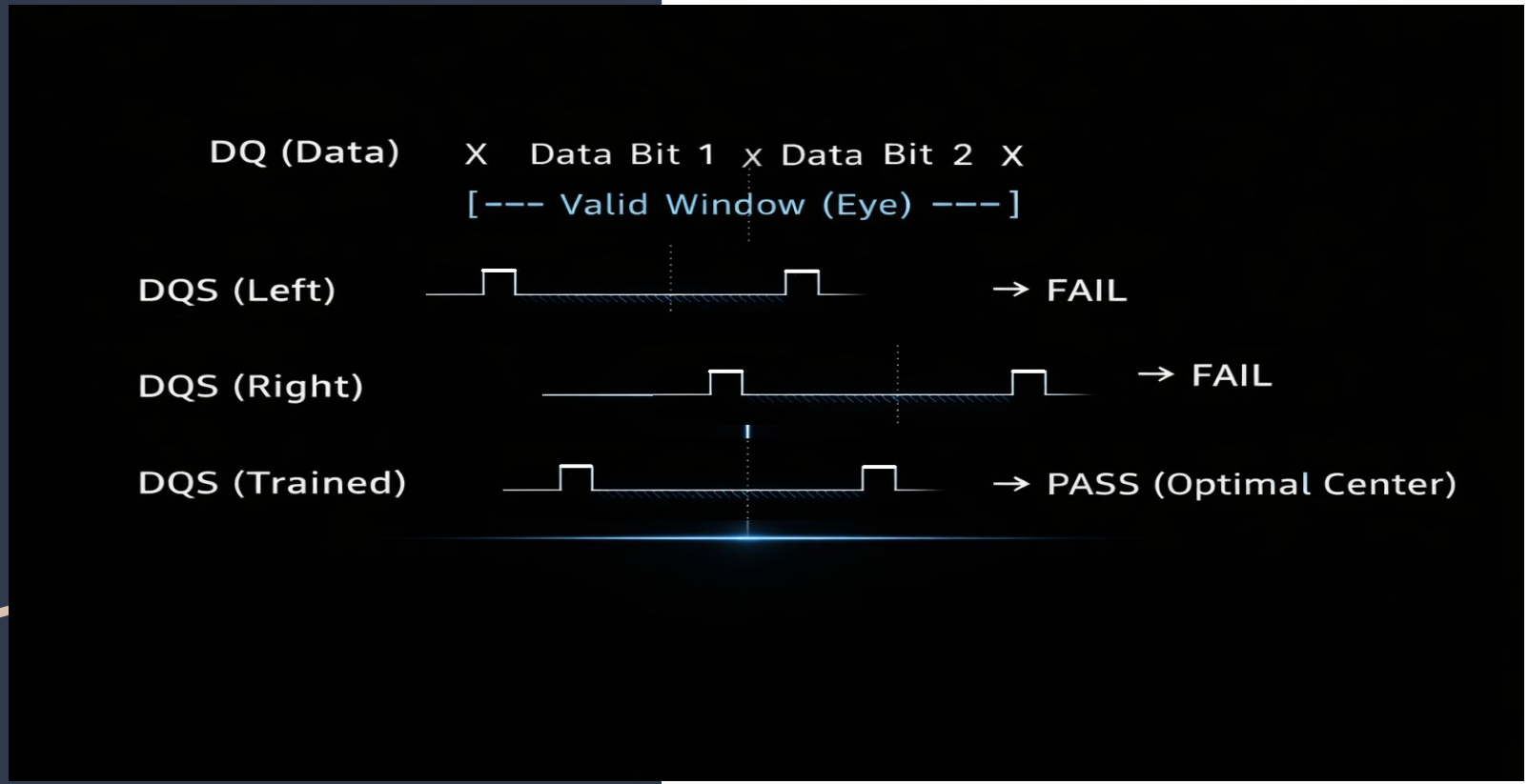
- It is a graphical tool used to analyze signal integrity
- The empty space in the middle of the diagram represents the "safe zone"
- Mask test : Signals should not overlap with the diamond in the centre

DQS Gate Training



Write Leveling

Read Data Eye Centering (DQ-DQS Centering)



DDR PHY Interface (DFI)

DFI acts as a "universal language" between the two components.

Advantages:

- We can mix and match components from different companies
- No need to redesign the interface every time vendor is switched
- Separate blocks will work together when integrated into the final chip

DFI 5.0/5.1

These three concepts are the "pillars" of **DFI 5.0/5.1**

1. **PHY independent boot sequence**
   - Controller was the boss in older versions
   - Controller sends a single "Start" command to the PHY

2. **Expanding frequency range support**
   - Hop between low and high speeds to save battery
   - Frequency Set Points
   - Smooth, high-speed performance

3. **New PHY-to-Controller Interface interaction**
   - Controller can send specific request packets to the PHY
   - synced to a "Write Clock" (WCK) that runs much faster than the Command clock
   - Interactions allow the PHY to instantly alert the Controller if it detects a "Command Address Parity" error